Adaptive Image Reads for Agent Training

Majd Zayyad

November 2026

Abstract

An agent-training job can leave its learner waiting for tools that are still fetching files from a remote image. Reading larger chunks reduces requests but can waste bandwidth; smaller chunks have the opposite cost. We study a reader that chooses how much to fetch from recent access patterns and observed request cost. The question is whether this local decision shortens the complete training job. We combine measured image-read jobs with a training scheduler that includes model calls, tool computation, queues, and learner updates. Across the calibrated read conditions, simulated training time is 24.6% lower when the other stages are short, and 0.6% lower with long model calls, relative to fixed256. The effect depends on where the job waits; losing stream identity can reverse it. The experiments explain when image-read savings reach the learner, and when other work absorbs them.

1 Why image reads matter

Agentic training repeatedly turns model output into actions: inspect a repository, import a library, compile a program, or run tests. Those tools need files inside their sandbox. With remote-backed images, some of the files arrive only when read. If tools finish late, a training update can wait even when the learner itself is ready.

DSec's microVM storage path illustrates this setting: it uses OverlayBD and fetches image data in 256 KiB chunks [2]. A fixed size is simple, but the useful amount of data differs between a short import, a scattered lookup, and a sequential scan. The opportunity is to change the fetch size without giving the reader more cache memory.

Read-ahead already adapts to sequential access, as in AMP [1]. TIP and Khameleon connect prefetching decisions to cost or expected usefulness [3, 4]. AdaCache selects aligned blocks for known missing intervals [6]; WaRP plans object-store requests from intervals already known to be required [5]. Our reader instead chooses a blocking refill before the next tool read is known. It combines likely continuation with the cost of the aligned windows that the current read would cross.

The endpoint matters as much as the reader decision: *does adaptive fetching reduce the time to finish the same amount of training work?* A faster image-read benchmark alone cannot answer that question.

2 Reader and training experiment

Choosing a refill. The reader considers aligned windows from 16 KiB to 1 MiB. Recent adjacent reads suggest that a stream will continue; jumps reduce that expectation. Completed requests update its estimate of fixed request cost and transfer rate. It selects the window with the lowest

$$\frac{\text{estimated fetch time}}{\text{bytes likely to be used}}.$$

The cost includes crossing multiple windows, not just the requested byte count. Cache capacity, replacement, workers, and in-flight limits are unchanged. Extra bytes arrive inside the blocking refill; there is no asynchronous prefetcher. The implementation and parameter definitions are in the accompanying code.

Measured tool-stage input. An independent C++ reader accesses a 512 MiB image over loopback HTTP with a 64 MiB cache. Random, sequential, and mixed traces are tested at three concurrency levels and under three controlled latency/bandwidth settings, giving 27 conditions. Each has three measured repetitions. Timing covers the entire replay process, including setup and drain. Only client-observable timings reach the controller.

All controls use the same executable. Alongside fixed256, we test other fixed sizes, choose one static setting on separate development traces, and include a simple grow/reset rule. That rule increases its window on successive missed demands and resets after a jump. Every measurement checks the returned data and common resource budgets.

From tool time to training time. We use each measured replay duration as the cost of one synthetic image-read job, not as a pure storage latency. A rollout consists of setup, a model call, the image-read job, additional tool computation, another model call, verification, and teardown. The simulator queues these stages behind limited actor, model, CPU, and tool-worker capacity. A learner update starts only after its assigned rollouts finish; the next update's rollouts wait for that update.

Every simulated run completes the same 256 rollouts and eight learner updates. Policies share job order, service-time variation, non-I/O work, and resources. We vary the cost of model calls, tool computation, and learner updates to expose different reasons a training job might wait. The clock stops after the final update, not after the last image read.

Training setting	Fixed256 (s)	Adaptive (s)	Time saved
Short compute stages	30.9	23.3	24.6%
Long model calls	662.0	658.0	0.6%
Long tool computation	353.3	342.6	3.0%
Long learner updates	214.1	193.6	9.6%

Table 1: Total simulated time through the final learner update: geometric means across 27 read conditions with identical work in every arm. These are model results, not measured GPU training runs.

Each tool worker has an independent reader/backend budget, and each image-read job starts with an empty reader cache. This is a controlled execution-time model: it does not simulate learning quality, real repository tasks, warm sandbox reuse, or contention inside a shared 3FS service. Complete parameters, individual outcomes, and descriptive intervals are provided with the data.

3 What changes total training time?

Table 1 reports the training endpoint rather than the isolated reader endpoint. With short compute stages, adaptive fetching gives 24.6% lower total time against fixed256. The development traces selected `fixed64`; against that static setting, training time is 11.2% lower. Against grow/reset it is 6.0% lower in the same setting.

Stream handling provides a more serious counterexample. In the sequential condition that collapses stream IDs behind a serialized frontend, simulated training takes 3.61× fixed256 time when other stages are short. Interleaved reads no longer look sequential to the policy; the read-job records show a shift toward small refills and more requests. The frontend setting is identical in both arms. This is a simulated failure case, not an observed slowdown of DSec.

These results make the deployment question more concrete. Image fetching deserves attention when it delays the supply of usable rollouts. When model calls, CPU work, or learner updates dominate, moving fewer bytes is not the same as finishing training much sooner. Preserving the identity of independent read streams is also part of the design, rather than a detail to leave to the storage integration.

The next step is to replace generated read jobs with captured coding-tool work and compare matched training runs. The current experiment answers the execution-time question under explicit assumptions; actual time-to-quality remains to be measured.

Code and reproduction. Source, calibration data, the full simulation matrix, and commands for recollecting the measurements and rebuilding this paper are public at https://github.com/majdzayyad/agent-training-image-io. Release `v1.0.0` fixes the version used here.

References

[1] Binny S. Gill and Luis Angel D. Bathen. AMP: Adaptive multi-stream prefetching in a shared cache. In *USENIX FAST*, pages 185–198, 2007. URL https://www.usenix.org/legacy/events/fast07/tech/gill.html.

[2] Jialiang Huang, Hongxuan Tang, Jingchang Chen, et al. DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale. arXiv:2609.22978v1, 2026. URL https://arxiv.org/abs/2609.22978v1.

[3] Haneen Mohammed, Ziyun Wei, Eugene Wu, and Ravi Netravali. Continuous prefetch for interactive data applications. *Proceedings of the VLDB Endowment*, 2020. doi: 10.14778/3407790.3407826. URL https://arxiv.org/abs/2007.07858v1.

[4] R. Hugo Patterson, Garth A. Gibson, Eka Ginting, Daniel Stodolsky, and Jim Zelenka. Informed prefetching and caching. In *ACM Symposium on Operating Systems Principles (SOSP)*, 1995. doi: 10.1145/224057.224064. URL https://www.pdl.cmu.edu/PDL-FTP/TIP/SOSP15.pdf.

[5] Ruipeng Wan, Xinyu Zeng, Yiming Qiao, and Huanchen Zhang. WaRP: Workload-aware request planning for vector retrieval on object storage. arXiv:2610.29689v1, public preprint, 2026. URL https://arxiv.org/abs/2610.29689v1.

[6] Qirui Yang, Runyu Jin, Ni Fan, Devasena Inupakutika, Bridget Davis, and Ming Zhao. AdaCache: A disaggregated cache system with adaptive block size for cloud block storage. In *IEEE International Conference on Cloud Computing (CLOUD)*, 2023. doi: 10.1109/CLOUD60044.2023.00048. URL https://arxiv.org/abs/2306.17254v1.